

Projeto Multidisciplinar — Trilha Back-end

Rede "Raízes do Nordeste" — API Back-end

Curso: Roteiro da Atividade Prática - Raízes do Nordeste - Trilha Back-end

Disciplina: Eletiva IV — Projeto Multidisciplinar de Análise e Desenvolvimento de Sistemas

Aluno: Albert Leonardo Belineli Freire

RU: 4711705

Polo de Apoio: Apucarana

Semestre: 2º semestre

Professor(es): Giuliano Lanes de Almeida (Esp.); Luciane Yanase Hibara Kanashiro (Me.); Rodrigo da Silva do Nascimento (Me.); Winston Sen Lun Fung (Me.)

Sumário

1. Introdução

2. Análise e Requisitos

3. Modelagem e Arquitetura

4. Entrega Técnica

5. Plano de Testes e Evidências

6. Conclusão

7. Referências

Anexo — Evidências Completas de Execução

Uso de Ferramentas de IA

1. Introdução

A rede "Raízes do Nordeste" nasceu como um pequeno negócio familiar em Recife e se expandiu para múltiplas capitais e cidades do interior, levando culinária nordestina ao público urbano através de diferentes canais de atendimento: aplicativo, totens de auto-atendimento, balcão e retirada rápida (pick-up). Esse crescimento acelerado trouxe desafios técnicos e organizacionais: necessidade de uma operação multicanal consistente, controle de estoque independente por unidade, conformidade com a LGPD no programa de fidelização, processamento de pagamentos desacoplado de provedores externos, e alta disponibilidade nos horários de pico.

Este documento apresenta a solução de back-end desenvolvida para resolver esses desafios: uma API REST que centraliza autenticação e autorização por perfil, gestão de pedidos multicanal, controle de estoque por unidade, programa de fidelidade com proteção de dados pessoais, e integração simulada com gateway de pagamento.

Objetivo do projeto: demonstrar, por meio de uma solução funcional, a capacidade de analisar um problema de negócio real, tomar decisões técnicas justificadas e construir uma arquitetura coerente, testável e auditável.

Principais usuários do sistema:

- Cliente — realiza pedidos, consulta fidelidade, acompanha status
- Atendente — visualiza pedidos e estoque, atualiza status de pedidos
- Cozinha — atualiza status de pedidos durante o preparo
- Gerente — controla estoque (entrada/saída) e supervisiona operação da unidade
- Admin — acesso irrestrito, gestão da rede como um todo

Relevância: o sistema serve de espinha dorsal tecnológica para a operação diária da rede, sendo o ponto de integração entre os canais de venda, o controle de estoque e a experiência de fidelização do cliente.

2. Análise e Requisitos

2.1 Requisitos Funcionais (RF)

ID	Requisito
RF01	Permitir cadastro de clientes, exigindo consentimento explícito LGPD
RF02	Autenticar usuários via e-mail/senha, emitindo token JWT
RF03	Controlar acesso por perfil — RBAC (CLIENTE, ATENDENTE, COZINHA, GERENTE, ADMIN)
RF04	Listar unidades ativas da rede
RF05	Exibir o cardápio de uma unidade considerando o estoque disponível naquele local
RF06	Permitir que o cliente crie pedidos multicanal (APP, TOTEM, BALCAO, PICKUP, WEB)
RF07	Validar disponibilidade de estoque antes de confirmar um pedido
RF08	Processar pagamento (simulado) e atualizar o status do pedido conforme o resultado
RF09	Permitir que a equipe da unidade consulte e atualize o status do pedido
RF10	Permitir controle de estoque por unidade (entrada/saída) pelo Gerente/Admin
RF11	Calcular e creditar pontos de fidelidade a cada pedido aprovado
RF12	Permitir que o cliente resgate pontos de fidelidade acumulados
RF13	Registrar auditoria de ações sensíveis (login, criação de pedido, cadastro)

2.2 Requisitos Não Funcionais (RNF)

ID	Requisito	Categoria
RNF01	Senhas armazenadas com hash bcrypt (salt 10), nunca retornadas em respostas	Segurança
RNF02	Autenticação stateless via JWT com expiração configurável (24h)	Segurança
RNF03	Consentimento explícito obrigatório e acesso restrito a dados de fidelidade de terceiros	LGPD
RNF04	Arquitetura em camadas (domínio isolado de	Manutenibilidade

	infraestrutura), permitindo trocar tecnologia de persistência sem alterar regras de negócio	
RNF05	Estoque e operações segregados por unidade, permitindo crescimento horizontal da rede	Escalabilidade
RNF06	Toda ação sensível registrada com usuário, IP e timestamp	Auditabilidade
RNF07	Contrato de API documentado (rota, autenticação, payload, respostas e erros possíveis)	Documentação
RNF08	Listagens paginadas para evitar sobrecarga em grandes volumes de dados	Desempenho
RNF09	Pagamento desacoplado via serviço mock, simulando integração com gateway externo real	Integração

2.3 Decisões técnicas justificadas

- **Pagamento mockado, não real:** conforme o estudo de caso, a Raízes do Nordeste não processa pagamentos diretamente — apenas solicita, recebe confirmação/negativa e atualiza o status. A implementação reflete isso com um serviço de pagamento simulado (PaymentService), que aprova ou recusa conforme a forma de pagamento informada (RECUSADO MOCK força recusa), sem acoplar a lógica de negócio a um gateway específico.
- **RBAC granular por rota:** cada endpoint sensível declara explicitamente os perfis autorizados (ex.: autorizar('ADMIN', 'GERENTE') para movimentação de estoque), tornando a regra de autorização auditável diretamente no código de rotas.
- **Erros controlados, não exceções genéricas:** cenários como falta de estoque (409) ou dados inválidos (422) retornam respostas estruturadas e previsíveis, em vez de falhas não tratadas — essencial para um sistema que não pode "cair" em horário de pico.

3. Modelagem e Arquitetura

3.1 Arquitetura em camadas

O sistema segue uma arquitetura em camadas inspirada em Clean Architecture, com o domínio de negócio isolado da infraestrutura:

```
src/  
├── domain/errors/           → Tipos de erro de negócio (AppError e especializações)  
├── application/use-cases/   → Regras de negócio (orquestram domínio + infraestrutura)  
├── infrastructure/  
│   ├── auth/               → Emissão e validação de JWT  
│   ├── database/prisma/    → Acesso a dados (Prisma Client)  
│   ├── logging/            → Logger estruturado (Winston) + serviço de auditoria  
│   └── payment/            → Gateway de pagamento mock  
└── api/  
    ├── controllers/        → Tradução HTTP <-> casos de uso  
    ├── middlewares/        → Autenticação (JWT) e autorização (RBAC)  
    ├── routes/              → Definição de rotas + documentação Swagger  
    └── swagger/             → Configuração OpenAPI
```

Uma requisição para criar um pedido, por exemplo, passa por: API (recebe e valida formato) → Application (orquestra: valida estoque, decide o fluxo) → Domain (regras: "este item está disponível?") → Infrastructure (persiste no banco, processa pagamento mock). Essa separação permite, por exemplo, substituir o PostgreSQL por outro banco sem que a lógica de cálculo de pedido ou de pontos de fidelidade seja afetada.

3.2 Diagrama de Casos de Uso

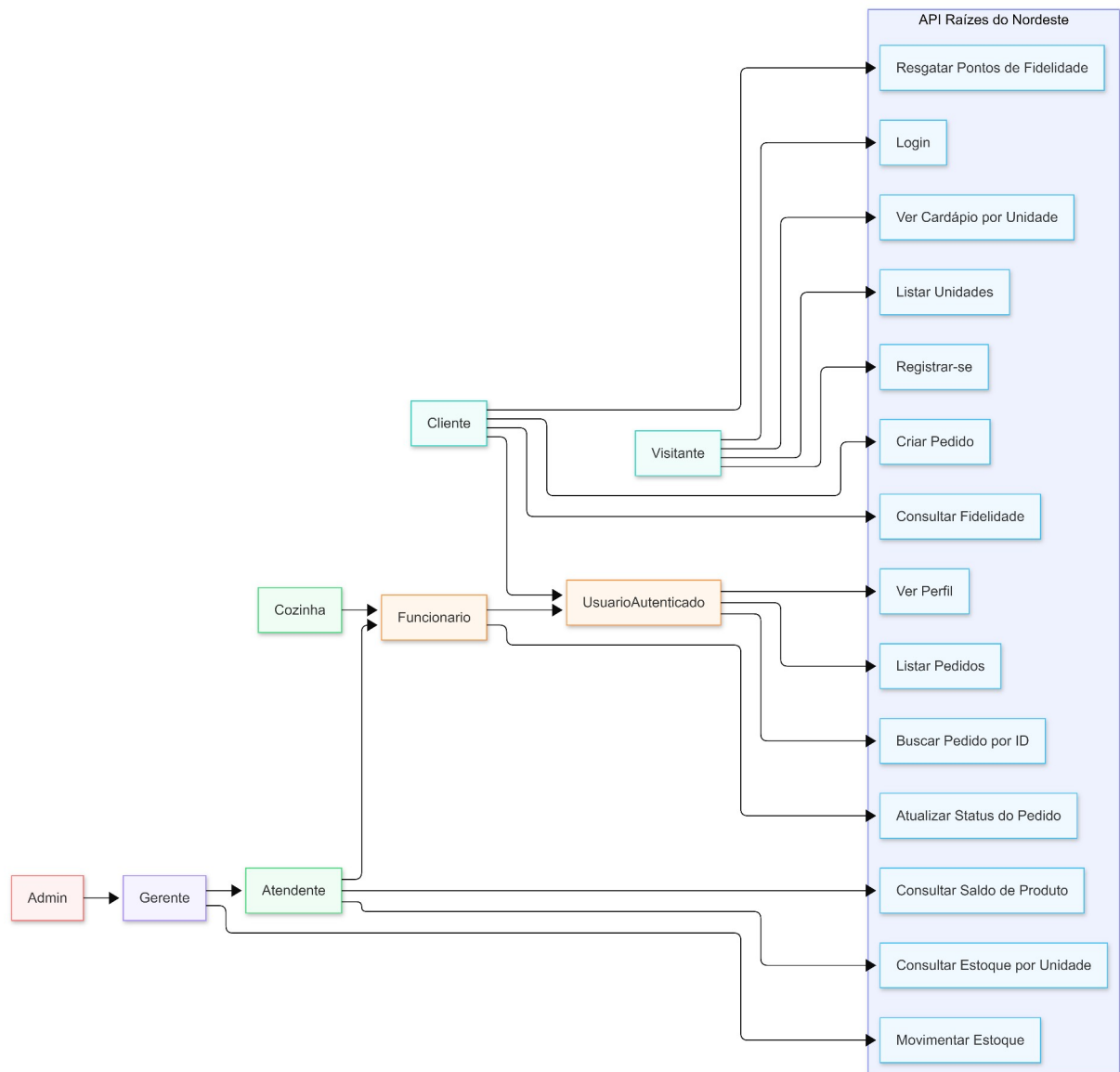


Figura 1 — Diagrama de Casos de Uso da API Raízes do Nordeste

Resumo da hierarquia de atores conforme RBAC implementado:

- Visitante (não autenticado): registrar-se, login, listar unidades, ver cardápio
- Usuário Autenticado (qualquer perfil): ver perfil, listar/buscar pedidos
- Cliente: criar pedido, consultar/resgatar fidelidade (apenas os próprios dados)
- Funcionário da Unidade (Atendente, Cozinha, Gerente, Admin): atualizar status do pedido
- Atendente/Gerente/Admin: consultar estoque e saldo por unidade
- Gerente/Admin: movimentar estoque (entrada/saída)

3.3 Diagrama de Classes

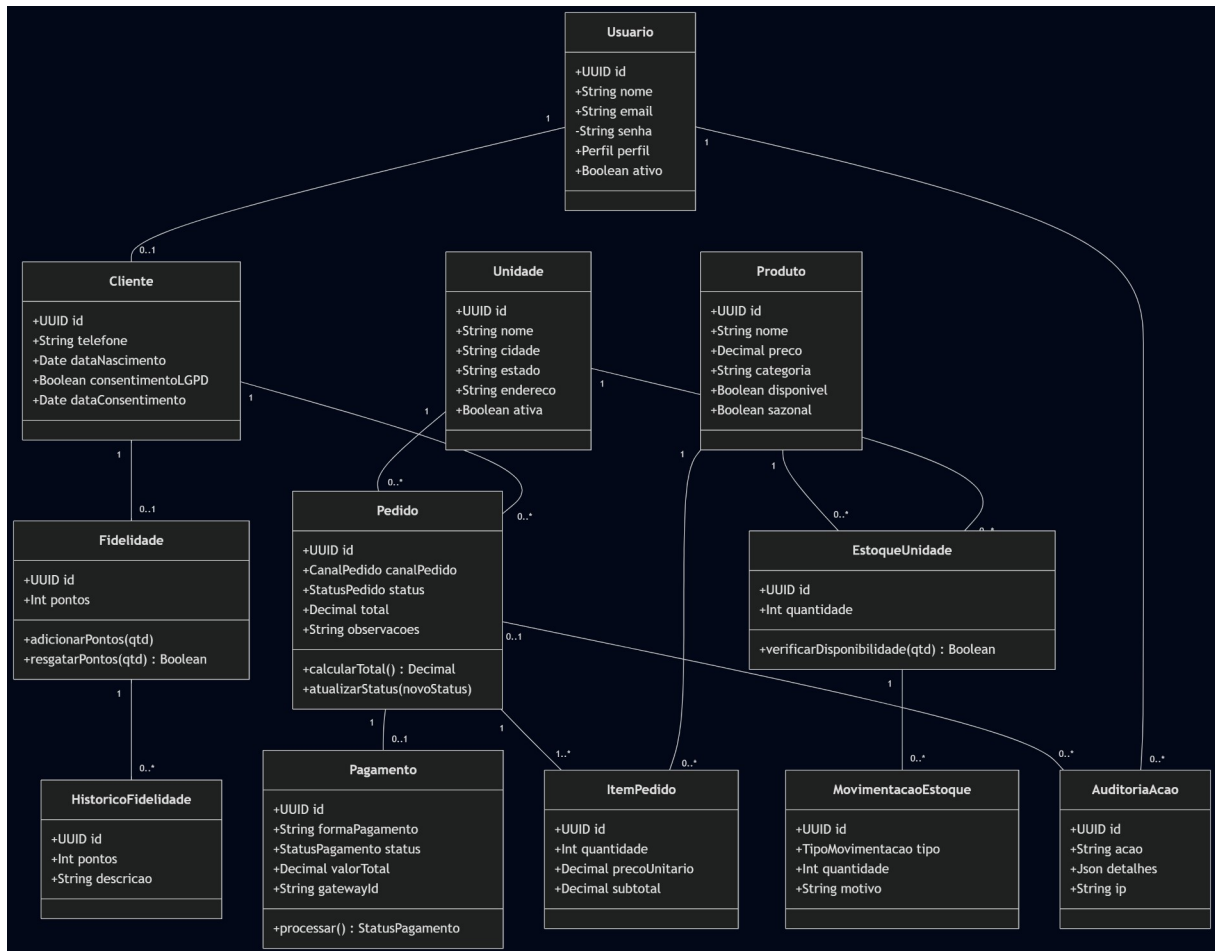


Figura 2 — Diagrama de Classes de domínio

Principais classes de domínio: Usuario, Cliente, Unidade, Produto, EstoqueUnidade, MovimentoEstoque, Pedido, ItemPedido, Pagamento, Fidelidade, HistoricoFidelidade e AuditoriaAcao, com seus relacionamentos e multiplicidades.

3.4 DER (Diagrama Entidade-Relacionamento)

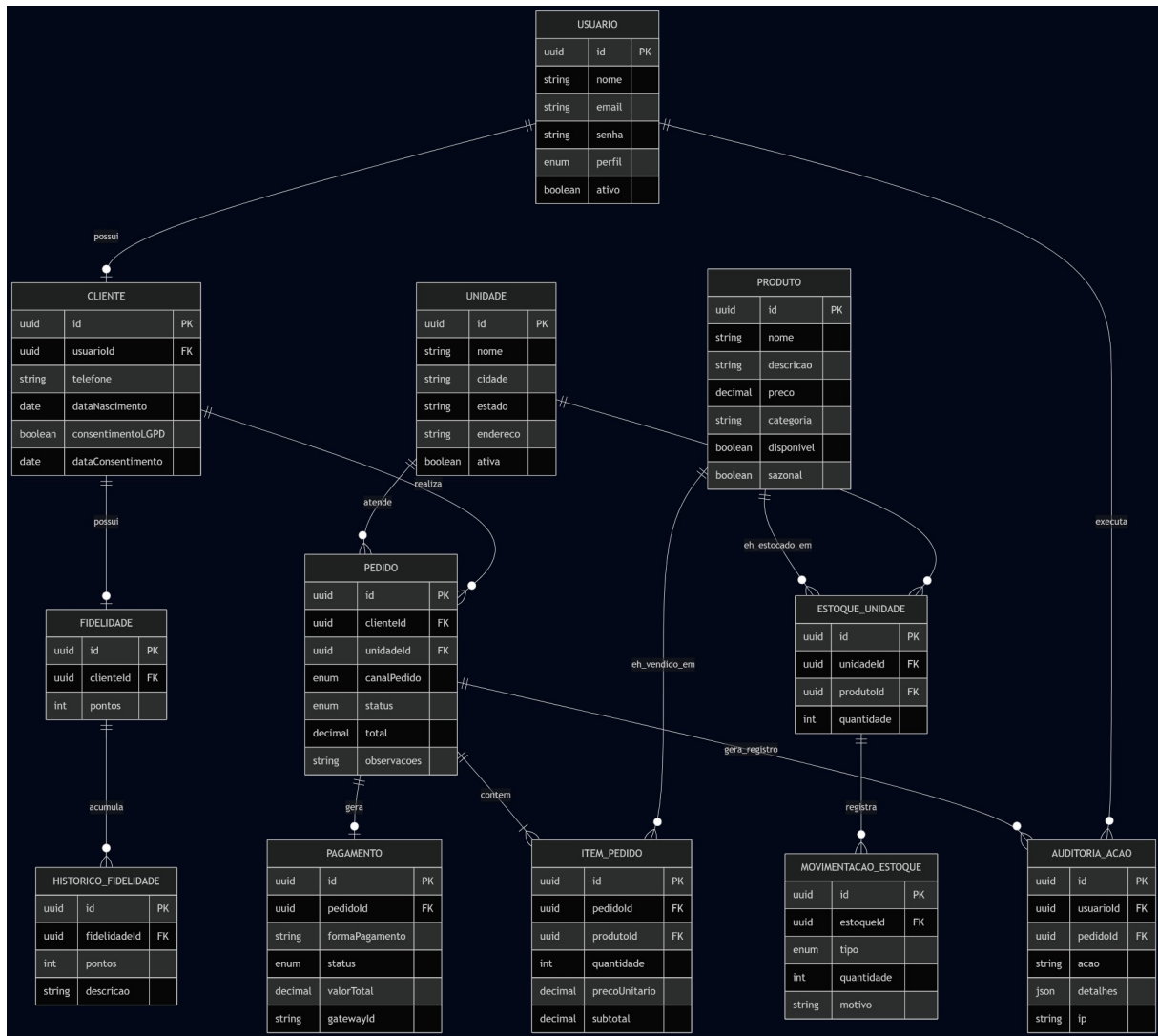


Figura 3 — DER do banco de dados PostgreSQL

O modelo persiste em 11 tabelas no PostgreSQL via Prisma ORM, refletindo fielmente as classes de domínio (ver prisma/schema.prisma no repositório).

3.5 Principais Endpoints da API

Método	Rota	Autenticação	Descrição	Possíveis erros
POST	/auth/registrar	Público	Cadastrar cliente (exige consentimento LGPD)	409, 422
POST	/auth/login	Público	Login e emissão de JWT	401
GET	/auth/perfil	JWT	Dados do usuário autenticado	401
GET	/unidades	Público	Listar unidades ativas	—
GET	/unidades/:id/cardapio	Público	Cardápio considerando estoque	404
POST	/pedidos	JWT + CLIENTE	Criar pedido (fluxo crítico)	401, 404, 409, 422
GET	/pedidos	JWT	Listar pedidos com filtros/paginação	401
GET	/pedidos/:id	JWT	Buscar pedido (CLIENTE só vê os próprios)	401, 403, 404
PATCH	/pedidos/:id/status	JWT + ADMIN/GERENTE/ATENDENTE/C OZINHA	Atualizar status do pedido	401, 403, 404
GET	/estoque/:unidadeId	JWT + ADMIN/GERENTE/ATENDENTE	Listar estoque por unidade	401, 403, 404
POST	/estoque/:unidadeId/movimentar	JWT + ADMIN/GERENTE	Movimentar estoque	401, 403, 422
GET	/fidelidade/:clienteId	JWT	Consultar fidelidade (CLIENTE só vê o próprio)	401, 403, 404
POST	/fidelidade/:clienteId/resgatar	JWT + CLIENTE	Resgatar pontos	401, 422

Documentação interativa completa (contrato de API) disponível via Swagger em </api/docs>.

3.6 Tecnologias

Node.js + TypeScript · Express · Prisma ORM + PostgreSQL · JWT (jsonwebtoken) · bcryptjs · Zod (validação) · Winston (logging estruturado) · Swagger (swagger-jsdoc + swagger-ui-express) · Jest (testes).

4. Entrega Técnica

- **Repositório:** <https://github.com/AlbertLeonardo34/raizes-nordeste>
- **Como rodar o projeto:** ver README.md na raiz do repositório (instalação, variáveis de ambiente, migrations, seed, execução)
- **Documentação interativa (Swagger):** <http://localhost:3000/api/docs>
- **Health check:** <http://localhost:3000/health>

5. Plano de Testes e Evidências

A estratégia de testes cobriu cenários funcionais positivos e negativos do fluxo crítico (criação de pedido), autenticação/autorização (RBAC), validação de dados e regras de LGPD. Os 14 cenários abaixo foram executados manualmente contra o servidor real. As evidências completas de request/response de cada cenário estão no Anexo deste documento (e também em tests/evidencias-execucao.md, no repositório).

ID	Cenário	Tipo	Resultado
T01	Login válido	Positivo	200 + accessToken
T02	Login com senha inválida	Negativo	401 CREDENCIAIS_INVALIDAS
T03	Acesso sem token	Negativo	401 NAO_AUTENTICADO
T04	Acesso com perfil sem permissão	Negativo	403 SEM_PERMISSAO
T05	Criar pedido válido (APP, PIX)	Positivo	201, status EM_PREPARO, pagamento APROVADO
T06	Criar pedido sem estoque suficiente	Negativo	409 CONFLITO
T07	Criar pedido sem canalPedido	Negativo	422 VALIDACAO_INVALIDA
T08	Pagamento mock recusado	Negativo	201, status CANCELADO, pagamento RECUSADO
T09	Listar pedidos filtrando por canal	Positivo	200 + lista paginada
T10	Atualizar status do pedido	Positivo	200 + statusAnterior/novoStatus
T11	Cardápio por unidade (considera estoque)	Positivo	200 + produtos disponíveis
T12	Movimentar estoque (entrada)	Positivo	200 + saldoAtual atualizado
T13	Registrar sem consentimento LGPD	Negativo	422 VALIDACAO_INVALIDA
T14	Consultar fidelidade própria	Positivo	200 + pontos + histórico

Resultado: 14/14 cenários aprovados (8 positivos, 6 negativos), confirmando que a API trata cenários de erro de forma controlada — sem falhas não tratadas — conforme exigido para um sistema que precisa permanecer disponível em horários de pico.

6. Conclusão

O desenvolvimento da API da Raízes do Nordeste evidenciou que decisões de arquitetura tomadas no início do projeto — especialmente a separação entre domínio e infraestrutura, e o tratamento de pagamento como um serviço externo desacoplado — facilitaram a evolução do sistema sem retrabalho. Durante os testes manuais, identificamos um problema real de integração: tokens JWT colados com formatação incorreta (aspas extras ou prefixo "Bearer" duplicado) eram rejeitados de forma genérica. A correção aplicada — tornar o middleware de autenticação tolerante a esses formatos comuns de erro de cliente — é um exemplo de decisão técnica fundamentada em um problema observado na prática, não hipotético.

Desafios enfrentados: garantir que o controle de estoque fosse consistente por unidade sem acoplar essa regra à camada de pedidos; modelar o programa de fidelidade respeitando a exigência de LGPD de que um cliente não acesse dados de outro.

Pontos de atenção para evolução futura: substituição do gateway de pagamento mock por uma integração real; introdução de cache para o cardápio em unidades de alto tráfego; uso de filas de mensagens para absorver picos de pedidos sem degradar o tempo de resposta; testes automatizados (Jest) cobrindo os mesmos cenários hoje validados manualmente.

7. Referências

- PRESSMAN, R. S.; MAXIM, B. R. Engenharia de Software: Uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.
- SOMMERVILLE, I. Engenharia de Software. 10. ed. São Paulo: Pearson, 2018.
- LEDUR, C. L. Análise e projeto de sistemas. 1. ed. Barueri: Manole, 2017.
- Documentação oficial: Express.js (expressjs.com), Prisma (prisma.io/docs), JWT (jwt.io), OpenAPI/Swagger (swagger.io).

Anexo — Evidências Completas de Execução

Execução real dos 14 cenários definidos no plano de testes, contra o servidor local (<http://localhost:3000/api/v1>). Todas as respostas abaixo são reais, capturadas via curl em 29/06/2026.

T01 — Login válido

Requisição: POST /auth/login {"email":"cliente@exemplo.com","senha":"Senha@123"}

STATUS: 200
{ "accessToken": "...", "tokenType": "Bearer", "expiresIn": 86400, "usuario":
{"perfil": "CLIENTE", ...} }

Esperado: 200 + accessToken — Confirmado.

T02 — Login com senha inválida

Requisição: POST /auth/login {"email":"cliente@exemplo.com","senha":"SenhaErrada"}

STATUS: 401
{ "error": "CREDENCIAIS_INVALIDAS", "message": "E-mail ou senha inválidos" }

Esperado: 401 + CREDENCIAIS_INVALIDAS — Confirmado.

T03 — Acesso sem token

Requisição: GET /pedidos (sem header Authorization)

STATUS: 401
{ "error": "NAO_AUTENTICADO", "message": "Token não fornecido" }

Esperado: 401 + NAO_AUTENTICADO — Confirmado.

T04 — Acesso com perfil errado (CLIENTE tentando movimentar estoque)

Requisição: POST /estoque/{unidadeId}/movimentar com token CLIENTE

STATUS: 403
{ "error": "SEM_PERMISSAO", "message": "Acesso restrito a: ADMIN, GERENTE" }

Esperado: 403 + SEM_PERMISSAO — Confirmado.

T05 — Criar pedido válido (canal APP, pagamento PIX)

Requisição: POST /pedidos com token CLIENTE, 2x Tapioca Simples

STATUS: 201
{ "pedidoId": "3abe5746-...", "status": "EM_PREPARO", "total": 25.8,
"pagamento": { "status": "APROVADO", "mensagem": "Pagamento aprovado com sucesso" } }

Esperado: 201 + status EM_PREPARO + pagamento APROVADO — Confirmado.

T06 — Criar pedido sem estoque suficiente (quantidade 999)

Requisição: POST /pedidos {"itens":[{"produtoId":"...", "quantidade": 999}]}

STATUS: 409

```
{"error":"CONFLITO","message":"Não há quantidade suficiente para um ou mais itens.",  
  "details":[{"field":"itens[0].quantidade","issue":"Disponível: 48"}]}
```

Esperado no plano original: ESTOQUE_INSUFICIENTE. Comportamento real: código CONFLITO — plano de testes corrigido para refletir o código real retornado pela API.

T07 — Criar pedido sem campo canalPedido

Requisição: POST /pedidos (sem canalPedido)

STATUS: 422

```
{"error":"VALIDACAO_INVALIDA","message":"Dados inválidos",  
  "details":[{"field":"canalPedido","issue":"canalPedido deve ser: APP, TOTEM, BALCAO,  
  PICKUP, WEB"}]}
```

Esperado: 422 + VALIDACAO_INVALIDA — Confirmado.

T08 — Pagamento mock recusado

Requisição: POST /pedidos {"formaPagamento":"RECUSADO MOCK"}

STATUS: 201

```
{"pedidoId":"e606e260-...", "status":"CANCELADO",  
  "pagamento":{"status":"RECUSADO","mensagem":"Pagamento recusado pela operadora"}}
```

Esperado: 201 + status CANCELADO + pagamento RECUSADO — Confirmado.

T09 — Listar pedidos filtrando por canal (GERENTE)

Requisição: GET /pedidos?canalPedido=APP

STATUS: 200

```
{"data":["/* 3 pedidos do canal APP */ ], "meta":  
{"total":3,"page":1,"limit":10,"totalPages":1}}
```

Esperado: 200 + lista paginada filtrada — Confirmado.

T10 — Atualizar status do pedido (GERENTE)

Requisição: PATCH /pedidos/{id}/status {"status":"PRONTO"}

STATUS: 200

```
{"pedidoId":"3abe5746-...", "statusAnterior":"EM_PREPARO", "novoStatus":"PRONTO"}
```

Esperado: 200 + statusAnterior + novoStatus — Confirmado.

T11 — Cardápio por unidade (considera estoque)

Requisição: GET /unidades/{id}/cardapio

STATUS: 200

```
{"unidade":{"nome":"Raízes do Nordeste - Recife Centro"},  
  "cardapio":["/* 7 produtos com quantidadeDisponivel */ ]}
```

Esperado: 200 + lista de produtos disponíveis — Confirmado.

T12 — Movimentar estoque (entrada, GERENTE)

Requisição: POST /estoque/{unidadId}/movimentar

`{"tipo":"ENTRADA","quantidade":10,"motivo":"Reposição semanal"}`

STATUS: 200

`{"saldoAtual":57, "movimentacao":{"tipo":"ENTRADA","quantidade":10}}`

Esperado: 200 + saldoAtual atualizado — Confirmado.

T13 — Registrar usuário sem consentimento LGPD

Requisição: POST /auth/registrar {"consentimentoLGPD": false, ...}

STATUS: 422

`{"error":"VALIDACAO_INVALIDA","message":"O consentimento LGPD é obrigatório para cadastro",
 "details":[{"field":"consentimentoLGPD","issue":"Deve ser true – LGPD exige consentimento
 explícito"}]}`

Esperado: 422 + VALIDACAO_INVALIDA — Confirmado.

T14 — Consultar fidelidade (cliente consultando os próprios pontos)

Requisição: GET /fidelidade/{clientId} com token do próprio cliente

STATUS: 200

`{"pontos":181,"historico":[
 {"pontos":25,"descricao":"Pedido 3abe5746... – +25 pontos"},
 {"pontos":6,"descricao":"Pedido c81ae4c3... – +6 pontos"}
]}`

Esperado: 200 + pontos + histórico — Confirmado.

Resumo: 14/14 cenários executados com sucesso (8 positivos, 6 negativos). O único ajuste necessário foi de nomenclatura no código de erro do T06 (CONFLITO em vez de ESTOQUE_INSUFICIENTE), corrigido no plano de testes. O comportamento de erro controlado (401/403/409/422) confirma que a API responde de forma previsível a cenários de falha, sem quebrar.

Uso de Ferramentas de IA

Conforme exigido pelo roteiro da atividade ("Uso de IA nesta atividade"), declaro de forma explícita o uso da ferramenta de IA Claude (Anthropic, via Claude Code) ao longo deste trabalho. O código-fonte da API (controllers, use-cases, middlewares, schema Prisma) foi desenvolvido por mim previamente. A IA foi utilizada como ferramenta de apoio nas seguintes etapas específicas, sem substituir minha compreensão do problema ou minhas decisões técnicas:

- Diagnóstico e correção de um bug real encontrado durante testes manuais: tokens JWT colados com aspas extras ou prefixo "Bearer" duplicado eram rejeitados. Prompt utilizado: "olha esse erro no terminal, esquisito" (com print do log). Trecho aproveitado: ajuste no middleware `src/api/middlewares/auth.middleware.ts` para normalizar o token antes de validar, e ajuste no texto de instrução do Swagger (`src/api/swagger/swagger.config.ts`).
- Geração das fontes dos diagramas UML (Caso de Uso, Classes, DER) em `docs/diagramas/`, a partir da leitura do código e do schema Prisma já implementados por mim. Prompt utilizado: pedido para analisar o código e propor os três diagramas exigidos pelo roteiro da trilha.
- Execução assistida dos 14 cenários do plano de testes (chamadas reais via curl contra o servidor local) e organização das respostas em `tests/evidencias-execucao.md` e no Anexo deste documento.
- Organização e redação do texto deste documento principal (Introdução, Requisitos, Arquitetura, Conclusão, Referências), com base na minha análise do estudo de caso e no código que eu já havia desenvolvido.

Em resumo: a implementação original da API é de minha autoria; a IA apoiou a correção de um bug pontual, a documentação visual (diagramas), a execução/organização de evidências de teste e a redação deste relatório.